MODULE 02 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

The AWS SDK and the API Request Lifecycle

What actually happens between your function call and the AWS service - and the five behaviours that surprise people.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 2 of 34

admin@sthithapragna.com

Every AWS service is an **HTTPS API**. The SDK is a convenience layer over it, and the convenience is where the surprises live.

What happens on one API call

01

Resolve

Work out the Region, the endpoint and the credentials to use.

02

Sign

Build a SigV4 signature over the request, using the secret key.

03

Send

Transmit over HTTPS and wait for a response.

04

Handle

Parse the response, or classify the error and decide whether to retry.

ON THE EXAM

Four stages, and a failure can come from any of them. Knowing which stage failed narrows a problem faster than reading the stack trace.

The SDKs

- Official SDKs for **Python, JavaScript, Java, .NET, Go, Ruby, PHP, C++, Rust, Kotlin and Swift**.
- All are generated from the same service definitions, so behaviour is consistent across them.
- Python's is **boto3**, built on **botocore**, which holds the API definitions as JSON.
- That generated design is why a new AWS API appears in the SDK almost immediately.
- Exam code snippets are usually **Python with boto3**, with some Java and JavaScript.

> ON THE EXAM
>
> You are not expected to write code in the exam. You are expected to read a snippet and say what it does or why it fails.

Constructing a client

- A **client** is created per service, and is where region and credentials are resolved.
- **Creating a client is not free.** It loads definitions and resolves credentials.
- So create clients **once**, outside a Lambda handler, and reuse them.
- Clients are generally thread-safe and designed to be shared.
- Creating one per invocation is a measurable and entirely avoidable cost.

> ON THE EXAM
>
> This is the single most repeated piece of Lambda advice, and it originates here: client construction is the expensive part, not the API call.

Which Region

- An explicit **region** argument on the client wins over everything.
- Then **AWS_REGION**, which Lambda sets automatically for you.
- Then the **region** setting in the shared config file, under the active profile.
- With none of those, the SDK raises an error rather than guessing.
- **IAM, CloudFront and Route 53 are global**, and their endpoints live in us-east-1.

> ON THE EXAM
>
> The last line explains a recurring confusion: a CloudFront or ACM call that must be made in us-east-1 regardless of where your application runs.

Which credentials

- The SDK walks a **provider chain** and uses the first source that returns credentials.
- Roughly: environment variables, then a web identity token, then the shared files, then the container endpoint, then the instance profile.
- Correctly written code therefore needs **no credential configuration at all**.
- Passing keys explicitly to a client constructor **bypasses the chain**, and is almost always wrong.
- Module 16 covers this properly, including why order matters.

> ON THE EXAM
>
> The chain is why the same code runs on a laptop, in Lambda and on EC2 without change. That portability is the whole point.

Configuration precedence

- **Explicit client configuration in code** beats everything else.
- Then **environment variables**, such as AWS_REGION or AWS_MAX_ATTEMPTS.
- Then the **shared config file**, under the profile named by AWS_PROFILE.
- A stray environment variable silently overrides your config file, with no warning.
- The same order applies to region, credentials, retries and endpoint settings alike.

> ON THE EXAM
>
> One order governs everything: code, environment, file. When behaviour differs between two machines running the same code, this is the first thing to check.

Signing

- Every request is signed with **SigV4**, over the method, URI, headers, payload hash and timestamp.
- **The secret key is never transmitted.** Only the derived signature travels.
- The signature is Region-specific and service-specific, which is why both must be resolved first.
- A clock more than about **5 minutes** out causes every call to fail on signature.
- The SDK does all of this. You sign manually only when calling AWS without an SDK.

> ON THE EXAM
>
> The clock skew case is worth remembering: a container or device with a wrong clock fails every call, and the error mentions the signature rather than the time.

Pagination

- Many list and query operations return a **truncated** result set, not everything.
- The response carries a **NextToken**, or IsTruncated, or LastEvaluatedKey depending on the service.
- **Ignoring it silently returns only the first page.** No error is raised.
- S3 ListObjectsV2 caps at 1,000 keys. DynamoDB Query and Scan cap at 1 MB.
- Code that works in development and misses records in production is usually this.

> READ THIS TWICE
>
> This is the most dangerous default in the SDK, because it fails silently and correctly. You get a valid response containing part of the answer.

Paginators

- A **paginator** wraps a truncated operation and iterates the whole result set for you.
- In boto3: **client.get_paginator('list_objects_v2')**, then iterate **paginate()**.
- Other SDKs offer the same idea as async iterators or auto-paginating methods.
- It handles the token, the loop and the termination condition. You handle the items.
- A manual loop works, and is one forgotten condition away from an infinite loop or a silent truncation.

> ON THE EXAM
>
> Paginators exist because the manual loop is easy to write and easy to write slightly wrong. Prefer them wherever the SDK offers one.

Waiters

- Many AWS operations return immediately while the work continues asynchronously.
- Starting an EC2 instance returns before it is running. Creating a table returns before it is ACTIVE.
- A **waiter** polls until a resource reaches a state, then returns.
- In boto3: **client.get_waiter('instance_running').wait(...)**.
- Waiters have sensible built-in intervals and a maximum attempt count, and are configurable.

> ON THE EXAM
>
> The alternative is a hand-written sleep-and-poll loop, which is where fixed delays and missing timeouts creep in. A waiter has both already.

Reading errors

- Service errors surface as exceptions carrying an **error code** and a message.
- In boto3, **ClientError** with **e.response['Error']['Code']** holding the code.
- Modelled exceptions exist too, such as **ConditionalCheckFailedException**.
- **Catch on the error code**, not on the message. Messages change; codes are contractual.
- The response metadata also carries the **request ID**, which AWS Support will ask for.

> ON THE EXAM
>
> Matching on message text is fragile and common. The error code is part of the API contract and is what you should branch on.

Retries, briefly

- The SDK **automatically retries** throttling and transient errors, with exponential backoff and jitter.
- So an error you actually see has usually been retried two or three times already.
- Wrapping SDK calls in your own retry loop **multiplies** the attempts rather than adding to them.
- Configure the SDK instead: retry mode and max attempts are both settings.
- Module 27 covers which errors are retryable and why jitter matters.

> ON THE EXAM
>
> Worth knowing from the start: the SDK is already retrying. A failure that reaches your code is not a one-off blip.

Idempotency tokens

- A retried creation call can create the resource **twice** if the first response was lost.
- Many create operations accept a **ClientToken** or similar idempotency parameter.
- AWS recognises a repeat with the same token and returns the original result.
- Some SDKs generate one automatically. Others require you to supply it.
- Without one, a timeout leaves you genuinely unsure whether the resource exists.

> ON THE EXAM
>
> The timeout case is the reason this exists. You never learn whether the request succeeded, so a blind retry is how duplicates get created.

Client or resource interface?

CLIENT

The full API

- One method per API operation
- Every service, every operation
- Returns dictionaries
- Paginators and waiters live here
- The one to learn

RESOURCE, BOTO3 ONLY

Object-oriented wrapper

- Objects with attributes and methods
- Only some services, some operations
- Reads more naturally
- In maintenance mode, no new features
- Reach for meta.client when it lacks something

THE TIEBREAKER

The resource interface is Python-only and no longer being extended. Learn the client interface, which exists in every SDK and covers everything.

Endpoints

- The SDK builds a regional endpoint from the service and Region: **service.region.amazonaws.com**.
- A custom **endpoint URL** can be set, for VPC endpoints, FIPS, dualstack or local testing.
- **Interface VPC endpoints** keep traffic on the AWS network without internet egress.
- **FIPS endpoints** exist where a compliance regime requires validated cryptography.
- A custom endpoint changes where the request goes, not what it is signed for.

> ON THE EXAM
>
> Overriding the endpoint is how local emulators are used in testing. The request is still signed as though it were going to the real service.

The five that surprise people

- **Results are paginated**, and ignoring the token silently returns one page.
- **Operations return before the work finishes**, which is what waiters are for.
- **The SDK is already retrying**, so your own loop multiplies attempts.
- **Environment variables beat your config file**, silently.
- **Client construction is the expensive part**, not the API call.

> ON THE EXAM
>
> Five defaults, and each produces behaviour that looks like a bug elsewhere. Knowing them is most of what this module is for.

Each of these is a default doing exactly what it is documented to do.

> A script lists the objects in an S3 bucket and reports 1,000 objects. The console shows the bucket contains roughly 47,000. The script raises no errors and the same code has been in production for months.
>
> It calls list_objects_v2 once and iterates the Contents array.

What is wrong?

A The IAM policy limits how many objects may be listed

B The script needs a longer timeout to retrieve all the objects

C The response is truncated at 1,000 keys and the continuation token is being ignored

D S3 eventual consistency means recent objects are not yet listed

◆ PAUSE AND DECIDE

What is wrong?

A **IAM limits the count**

IAM grants or denies an action. It has no concept of allowing a limited number of results.

B **Longer timeout**

The call completes successfully and quickly. Nothing is timing out.

C **Truncated at 1,000 keys**

ListObjectsV2 returns at most 1,000 keys per call, with IsTruncated and a continuation token. Ignoring them returns exactly one page, silently.

D **Eventual consistency**

S3 has had strong read-after-write consistency since December 2020, and 46,000 missing objects is not a consistency window.

THE SIGNAL

Exactly 1,000 is the tell. A round number matching a documented page size is a pagination limit, not a coincidence.

> A deployment script creates a DynamoDB table and immediately writes seed data into it. The write intermittently fails with ResourceNotFoundException, more often on larger tables.
>
> The team has added a fixed 10-second sleep after table creation, which usually works.

What is the correct fix?

A Use the table_exists waiter after CreateTable, before writing

B Increase the sleep to 60 seconds

C Catch the exception and retry the write in a loop

D Create the table in a separate script run earlier

◆ PAUSE AND DECIDE

What is the correct fix?

A The table_exists waiter

CreateTable returns while the table is still CREATING. A waiter polls until it is ACTIVE, with a sensible interval and a maximum attempt count already built in.

B Longer sleep

Trades correctness for a guess. Too short still fails, too long wastes time on every run, and neither is reliable.

C Catch and retry

Rebuilds a waiter by hand, and a naive loop has no backoff and no maximum, so a genuinely missing table loops forever.

D Separate earlier run

Avoids the race by splitting the script, and the same race returns whenever the two runs are close together.

THE SIGNAL

A fixed sleep is the tell that a waiter is missing. The operation returned before the work finished, which is exactly the case waiters exist for.

> A developer runs a deployment script that works on their colleague's machine but fails on theirs, writing to the wrong AWS account. Both have identical code and an identical ~/.aws/config naming the same profile.
>
> The developer previously exported credentials into their shell to debug an unrelated issue.

What is the most likely cause?

A The shared config file is corrupt and must be recreated

B The two machines are running different SDK versions

C The profile in the config file names a different Region

D Exported environment variables take precedence over the shared config file

◆ PAUSE AND DECIDE

What is the most likely cause?

A **Corrupt config file**

The scenario states the config files are identical, and a corrupt file would produce a parse error rather than silently using another account.

B **Different SDK versions**

Precedence order is consistent across SDK versions and languages. It is not a version-dependent behaviour.

C **Different Region**

A wrong Region writes to the wrong Region, not the wrong account. The symptom names the account.

D **Environment variables win**

Code beats environment beats file. Exported credentials silently override the profile, and remain in the shell until it is closed.

THE SIGNAL

The last sentence is the planted clue. Exported credentials outlive the task they were exported for, and nothing warns you they are being used.

> A provisioning service creates infrastructure on customer request. Occasionally two identical resources appear for one request. Logs show the creating API call timed out from the client's perspective, and the SDK retried it.
>
> The operation supports a ClientToken parameter, which the code does not currently supply.

What should be changed?

A Disable SDK retries for this call by setting max attempts to 1

B Supply a ClientToken derived from the customer request, so a retry is recognised as a repeat

C Increase the client timeout so the call does not time out

D Check whether the resource exists before creating it

◆ PAUSE AND DECIDE

What should be changed?

A Disable retries

Removes duplicates by removing resilience. A genuinely transient failure now fails the customer request outright.

B Supply a ClientToken

AWS recognises a repeat carrying the same token and returns the original result rather than creating a second resource. Retries stay safe.

C Longer timeout

Reduces how often it happens without removing the case. A timeout can still occur, and the retry still duplicates.

D Check before creating

A check-then-act race. Two concurrent requests can both see nothing and both proceed, which is the bug idempotency tokens remove.

THE SIGNAL

The scenario names the parameter, which is the giveaway. Options A and C both make retries less likely rather than making them safe.

What to carry forward

- **Every service is an HTTPS API.** The SDK resolves, signs, sends and handles.
- **Results are paginated.** Use a paginator, and never assume one call returned everything.
- **Operations return before the work finishes.** Use a waiter rather than a sleep.
- Precedence is code, then environment, then file, for every setting alike.
- Create clients once and reuse them. Construction is the expensive part.

> ON THE EXAM
>
> Next: AWS Lambda, which is where most of this course lives - and where every one of these behaviours shows up again.